

Introduction to Unix shell

Part 5

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Swiss army knives pt.1

- Shell is often used to manage text files
 - Configuration files
 - Log files
 - Tabular data (e.g., `.csv` files)
- Commands useful to this aim
 - `grep`: text search
 - `sed`: stream editor for manipulating files
 - `awk`: stream editors for manipulating structured data and text.
- Useful format for pattern matching and substitution: **regular expression** (Regex)

- Regular expression: pattern used to match text
- Provides a powerful way to search for and manipulate strings
- Supported by many Unix tools: `grep`, `sed`, `awk`, etc.
- Two main types of regular expressions
 - **Basic Regular Expressions (BRE)**: Limited syntax, used by default in `grep`
 - **Extended Regular Expressions (ERE)**: Richer syntax, used with `grep -E` or `egrep`
- Perl Compatible Regular Expressions (PCRE): Most powerful, used with `grep -P`

- `.`: Matches any single character (except newline)
 - Example: `c.t` matches `cat`, `cut`, `c3t`
- `*`: Matches zero or more of the preceding character
 - Example: `ca*t` matches `ct`, `cat`, `caat`, `caaat`
- `^`: Matches the beginning of a line
 - Example: `^hello` matches lines starting with `hello`
- `$`: Matches the end of a line
 - Example: `world$` matches lines ending with `world`
- `[ ]`: Character class; matches any single character within the brackets
 - Example: `[aeiou]` matches any vowel
- `[^ ]`: Negated character class; matches any character NOT in the brackets
 - Example: `[^0-9]` matches any non-digit character

- **+**: Matches one or more of the preceding character (ERE only)
 - Example: `ca+t` matches `cat`, `caat`, `caaat` (not `ct`)
- **?**: Matches zero or one of the preceding character (ERE only)
 - Example: `colou?r` matches `color` and `colour`
- **{n,m}**: Matches between `n` and `m` occurrences (ERE only)
 - Example: `a{2,4}` matches `aa`, `aaa`, `aaaa`
- **|**: Alternation; matches either pattern (ERE only)
 - Example: `cat|dog` matches either `cat` or `dog`
- **()**: Grouping; treats multiple characters as a single unit (ERE only)
 - Example: `(ab)+` matches `ab`, `abab`, `ababab`

- `[a-z]`: Lowercase letters
- `[A-Z]`: Uppercase letters
- `[0-9]`: Digits
- `[a-zA-Z0-9]`: Alphanumeric characters
- `[[:alpha:]]`: All letters (equivalent to `[a-zA-Z]`)
- `[[:digit:]]`: All digits (equivalent to `[0-9]`)
- `[[:alnum:]]`: All alphanumeric characters
- `[[:space:]]`: Whitespace characters (space, tab, newline)
- `[[:upper:]]`: Uppercase letters
- `[[:lower:]]`: Lowercase letters
- `[[:punct:]]`: Punctuation characters

- By default, `grep` uses Basic Regular Expressions (BRE)
- Examples
 - `grep "^ERROR" log.txt`: Lines starting with ERROR
 - `grep "\.log$" files.txt`: Lines ending with .log
 - `grep "[0-9]" data.txt`: Lines starting with a digit
 - `grep "[aeiou]" words.txt`: Lines containing any vowel
 - `grep "colou*r" text.txt`: Matches color, colour, colourur, etc.
- **NOTE:** In BRE, special characters like +, ?, | need to be escaped with \
 - Example: `grep "ca\+t"` for one or more a

- Use `grep -E` (or `egrep`) for Extended Regular Expressions (ERE)
- Special characters do NOT need escaping in ERE
- Examples
 - `grep -E "cat|dog" text.txt`: Lines containing cat or dog
 - `grep -E "^[0-9]+$" numbers.txt`: Lines with only digits
 - `grep -E "^[a-z]+@[a-z]+\.[a-z]+$" emails.txt`: Email pattern (simple)
 - `grep -E "(abc)+" text.txt`: One or more repetitions of abc

- **IP Address:** `[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}`
- **Email:** `[a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z]{2,}`
- **URL:** `https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`
- **Date (YYYY-MM-DD):** `[0-9]{4}-[0-9]{2}-[0-9]{2}`
- **Time (HH:MM):** `[0-9]{2}:[0-9]{2}`
- **Word at end of line:** `word$`
- **Repeated word:** `([a-z]+) \1` (requires capture groups)
 - `([a-z]+)` matches a word and marks it as a group
 - `\1` is a reference to the previous group

Practical grep with Regex Examples

```
# Find all IP addresses in a log file
grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log
# Find all lines with HTTP status codes (3 digits)
grep -E "[0-9]{3}" access.log
# Find failed login attempts (case-insensitive)
grep -iE "failed|error|denied" auth.log
# Extract lines with email addresses
grep -oE "[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" data.txt
# Find lines that do NOT start with a comment
grep -E "^[^#]" config.txt
# Find lines with URL-like patterns
grep -oE "https?://[a-zA-Z0-9.-]+" text.txt
# Count lines with specific patterns
grep -cE "success|completed" log.txt
```

Log File Analysis with Regex

```
# Apache access.log format:
# IP_ADDRESS - - [DATE] "REQUEST" STATUS_CODE SIZE
# Extract successful requests (status 2xx)
grep -E " 2[0-9]{2} " access.log
# Extract client errors (status 4xx)
grep -E " 4[0-9]{2} " access.log
# Extract server errors (status 5xx)
grep -E " 5[0-9]{2} " access.log
# Count all error responses (4xx and 5xx)
grep -cE " [45][0-9]{2} " access.log
# Find requests that took more than 1 second (hypothetical)
grep -E " [0-9]{3} [0-9]{4,}" access.log
# Find POST and PUT requests
grep -E "(POST|PUT)" access.log
# Find requests to .php files
grep -E "\.php" access.log
```

- `^`: Start of line
 - `grep "[a-z]" file.txt` finds lines starting with lowercase letter
- `$`: End of line
 - `grep ";$" file.txt` finds lines ending with semicolon
- `\<`: Start of word (PCRE)
 - `grep -P "\<the\>" file.txt` matches word “the” only
- `\>`: End of word (PCRE)
- `\b`: Word boundary (PCRE)
 - `grep -P "\bcat\b" file.txt` matches “cat” but not “category”

- Special regex characters must be escaped with `\` when used literally
- Common characters to escape: `.` `*` `+` `?` `[` `]` `(` `)` `{` `}` `|` `^` `$`
- Examples
 - Match literal dot: `grep "\.txt"` (matches `.txt`, not any character)
 - Match literal question mark: `grep "what\?"`
 - Match literal asterisk: `grep "\*"`
 - Match literal parenthesis: `grep "func\(\)"`
- When using `grep` with patterns in shell, be careful with quoting
 - Single quotes prevent shell interpretation: `grep '[0-9].*\.txt$' file`
 - Double quotes allow variable expansion but may require escaping sequences

- **sed**: stream editor for filtering and transforming text
 - sed = **s**tream **e**ditor
 - Useful for text substitution, deletion, insertion, and transformation
- Reads input line by line, applies specified operations, and outputs the result
- Basic usage: **sed** [**options**] '**<script>**' **<file>**
- Common options
 - **-e <script>**: Specify an editing script (can be used multiple times)
 - **-f <file>**: Read script from a file
 - **-i**: Edit files in-place (modifies the original file)
 - **-i.bak**: Create a backup before editing in-place
 - **-n**: Suppress automatic printing of pattern space
 - **-r** or **-E**: Use extended regular expressions

- **s**: Substitution (find and replace)
 - Format: `s/pattern/replacement/[flags]`
- **d**: Delete lines matching a pattern
 - Format: `/pattern/d`
- **p**: Print lines matching a pattern
 - Format: `/pattern/p` (use with `-n` flag)
- **a**: Append text after a line
 - Format: `/pattern/a text`
- **i**: Insert text before a line
 - Format: `/pattern/i text`
- **c**: Change (replace) entire lines
 - Format: `/pattern/c text`
- **y**: Transliterate characters
 - Format: `y/source/dest/` (source and dest must be same length)

Substitution Flags in sed

- **g**: Global substitution (replace all occurrences on a line)
 - Default is to replace only the first occurrence
- **p**: Print lines where substitution occurred
- **i** or **I**: Case-insensitive matching
- **<number>**: Replace only the **<number>**-th occurrence
 - Example: **s/foo/bar/2** replaces the second occurrence
- **w <file>**: Write lines where substitution occurred to a file

- Addresses specify which lines the command applies to
- Address types
 - Line number: `5s/old/new/` (applies to line 5 only)
 - Range of lines: `5,10s/old/new/` (applies to lines 5-10)
 - Pattern matching: `/pattern/s/old/new/` (lines matching pattern)
 - Pattern range: `/start/,/end/s/old/new/` (from first match of start to end)
 - `$`: Last line, e.g., `$s/old/new/` replaces on the last line only
- Negation: `!` inverts the address
 - Example: `/pattern/!s/old/new/` applies to all lines NOT matching pattern

Basic sed Examples

```
# Simple substitution (first occurrence only)
echo "foo bar foo" | sed 's/foo/baz/' # Output: baz bar foo
# Global substitution (all occurrences)
echo "foo bar foo" | sed 's/foo/baz/g' # Output: baz bar baz
# Case-insensitive substitution
echo "Foo bar F00" | sed 's/foo/baz/gi' # Output: baz bar baz
# Substitute on specific line
sed '2s/old/new/' file.txt # Only line 2 is affected
# Substitute on lines 3-5
sed '3,5s/old/new/g' file.txt # Only lines 3, 4, and 5 are affected
# Substitute lines matching a pattern
sed '/error/s/~/[ERROR] /' log.txt # Prepend "[ERROR] " to lines containing "error"
```

Deletion and Printing Examples

```
# Delete all blank lines
sed '/^$/d' file.txt
# Delete lines matching a pattern
sed '/error/d' log.txt # Removes all lines containing "error"
# Delete lines NOT matching a pattern
sed '/success/!d' log.txt # Keeps only lines containing "success"
# Print specific lines (use -n to suppress default output)
sed -n '5,10p' file.txt # Print lines 5-10 only
# Print lines matching a pattern
sed -n '/ERROR/p' log.txt # Print only lines containing "ERROR"
# Print with line numbers
sed -n '/pattern/= ' file.txt # Shows line numbers of matching lines
```

Insertion, Append, and Change Examples

```
# Append text after matching lines
sed '/error/a [END OF ERROR MESSAGE]' log.txt
# Insert text before matching lines
sed '/error/i [START OF ERROR MESSAGE]' log.txt
# Change entire lines matching a pattern
sed '/error/c [LINE REPLACED]' log.txt
# Add text at the beginning of a file
sed '1i ===== FILE START =====' file.txt
# Add text at the end of a file
sed '$ a ===== FILE END =====' file.txt
# Insert on line number
sed '5i \New line inserted here' file.txt # Inserts new line before line 5
```

```
# Multiple substitutions with -e
sed -e 's/old/new/g' -e 's/foo/bar/g' file.txt
# Character transliteration (convert to uppercase)
sed 'y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/' file.txt
# Edit file in-place
sed -i 's/old/new/g' file.txt # Original file is modified
# Edit file in-place with backup
sed -i.bak 's/old/new/g' file.txt # Original saved as file.txt.bak, file.txt is
    modified
# Use extended regex with -E: converts phone format from 123-4567 to 123.4567
sed -E 's/([0-9]{3})-([0-9]{4})/\1.\2/g' file.txt
# Use alternation with -E
sed -E 's/(cat|dog)/[ANIMAL: \1]/g' file.txt # Marks occurrences of cat or dog
```

```
# Remove leading/trailing whitespace
sed 's/^[[[:space:]]*//; s/[[[:space:]]*$// ' file.txt
sed -e 's/^[[[:space:]]*//' -e 's/[[[:space:]]*$//' file.txt
# Convert Windows line endings to Unix
sed 's/\r$//' file.txt # Removes carriage return characters
# Add line numbers
sed 's/^/NR. &/' file.txt
# Remove comments (lines starting with #)
sed '/^[[[:space:]]*#/d' config.txt
# Highlight specific patterns in output
sed 's/ERROR/>>ERROR<</' log.txt
# Extract lines between two patterns
sed -n '/START/,/END/p' file.txt
# Replace tabs with spaces
sed 's/\t/ /g' file.txt
# Remove duplicate blank lines (keep single blank lines)
sed '/^$/N;/~\n$/!P;D' file.txt
```

Lab experience

- Given a web server access.log file count the number of requests that are successful (200) or wrong (4xx and 5xx) using `grep` and `wc`
- Analyze the size of the response
- Data Cleaning and Transformation with `sed`: make consistent a file where commas and tabs are used interchangeably as field delimiters.
- Extract all email addresses from a text file.

Example of a log record combined log format

```
83.149.9.216 - - [17/May/2015:10:05:03 +0000]
```

```
"GET /kibana-search.png HTTP/1.1" 200 2030 "http://semicomplete.com/"
```

```
"Mozilla/5.0 AppleWebKit/537.36 Chrome/32.0.1700.77 Safari/537.36"
```

- Client IP Address
- Remote logname + user (usually - -)
- Date
- Requeste line: "HTTP Method + URL + HTTP version"
- Reponse code (es: 200, 404, 502)
- Size
- Referrer URL
- Browser signature

Getting the list of response codes

```
#!/bin/bash
# find distinct response codes
F="access.log"
printf "found response code: %s\n" $(cut -d" " -f9 $F | sort -nu | tee respcodes.txt)
```

Code explanation

- `cut -d" " -f9 $F`: set delimited as space; get 9th field (field number obtained counting spaces)
- `sort -nu`: sort numerically and get unique values
- `tee respcodes.txt`: save on file and send to output

```
#!/bin/bash
# find distinct response codes
F="access.log"
printf "found response code: %s\n" $(grep -oP '" \K\d{3}(?= )' $F | sort -nu | tee
    respcodes.txt)
```

- `grep -oP '" \K\d{3}(?= )' $F`
 - `grep -oP '...':` use Perl regexp; report only the matching part
 - `" \K:` match a space after a `"`; discard this match
 - `\d{3}` match 3 digits (response code)
 - `(?= ):` match a space after the 3 digits, but do not include it in the match

Getting count of response codes

```
#!/bin/bash
ACC="access.log"
RC="respcodes.txt"
while read CODE
do
    N=$(grep "\" ${CODE} " ${ACC} | wc -l)
    printf "Resp Code: %d -> # occurrences: %d\n" ${CODE} $N
done < "$RC"
```

Code explanation

- `while read CODE ... done < $RC`: iterate over file \$RC, read line-by-line and store response code in CODE
- `grep "\" ${CODE} " ${ACC}`: get lines with response code
- `wc -l`: count lines

Other possible implementations

- With a `for` loop
- Single pass, identify and count respcodes using an associative array

```
F=access.log
NLINES=5
echo "* using cut"
cut -d" " -f10 $F | sort -rn | head -n $NLINES
echo "* using RegExp"
grep -Eo '[[[:space:]][[[:digit:]]{3}[[[:space:]][[[:digit:]]]+' $F | cut -f3 -d' ' |
    sort -rn | head -n $NLINES
echo "* using Perl RegExp"
grep -Po '(?<=\s\d{3}\s)\d+' $F | sort -rn | head -n $NLINES
```

- Top 5 largest requests
- Using `cut`
 - `cut -d" " -f10` 10th: field is the size
 - `sort -rn`: Sort numerically
 - `head -n $NLINES`: show only first NLINES elements

- Using regexp
 - `grep -Eo` uses regexp; output only matched part
 - `[:space:][:digit:]3[:space:]` matches space, 3 digits, a space (response code)
 - `[:digit:]+` matches a number (size)
 - Using `cut` to remove the response code
- Using Perl regexp
 - `grep -Po` uses Perl regexp; output only matched part
 - `(?<= ... )`: a look-behind expression, do not show in output
 - `=\s\d3\s`: matches space, 3 digits, a space (response code)
 - `\d+`: matches a number (size)

- A .csv file can contain lots of errors
- Sample file on horse breeds
 - Breed name, height and weight (average and std deviation)
- Types of errors
 - Missing " in strings
 - Missing , between fields
 - Multiple spaces

Example:

```
"Frisone", 160, 4, 600, 40
Connemara, 145, 4      400, 30
"Belga da Tiro",      172, 7, 900, 75
```



```
#!/bin/bash
# rely on GNU sed. Not working on BSD!
FIN="horse_breeds.csv"
FOUT="horse_breeds_clean.csv"
sed -E '
    # Add " to first column if missing
    s/^(("[,]+)"/"\1"/;
    # spaces between numbers becomes ","
    :a; s/([0-9])\s+([0-9])/ \1, \2/g; ta;
    # only one space after comma (" , " -> ", ")
    s/, \s*/ , /g;
    # Remove spaces at the end of the line
    s/\s+$//
' $FIN | tee $FOUT
```

- `s/^(~",]+)"\1"/;`
 - Match at the beginning of the line any character that is not " or ,
 - Mark the match as a group
 - Enclose the group between "
- `s/, \s*/ , /g;`
 - Match a comma (,) followed by zero or more spaces
 - Change it to a comma separated by exactly one space
 - Can occur more than once per line (g modifier)
- `s/\s+$//;`
 - Match a one or more spaces at the end of the line
 - Remove them

- `:a; s/([0-9])\s+([0-9])/\1, \2/g; ta;`
 - `([0-9])\s+([0-9])`: Match a number followed by one or more spaces and then a number (it is a field separator without the comma ,)
 - Mark the first and last digit as groups
 - `\1, \2`: Preserve the two groups, add comma and space in between
 - Can occur more than once per line (g modifier)
 - `:a; ...; ta`: iterate repeating the substitution until no match occurs
- Why the iteration
 - The match consumes two numeric fields
 - Two consecutive errors determines a match only for the first one
- Consider the case of input 1 2 3 4
 - The first pass matches 1 2 and 3 4
 - The result would be 1, 2 3, 4
 - The wrong sequence 2 3 remains

- `file`: Determines file type by reading file content (magic bytes)
 - Does NOT rely on file extension
 - Checks file headers/signatures to identify type
- Basic usage: `file <filename>`
- Common options
 - `-b`: Brief mode (omit filename in output)
 - `--mime-type`: Output MIME type only
 - `-i`: Same as `--mime-type`
 - `-r`: Do not follow symbolic links

- Examples of magic signatures (file headers)
 - PNG: 89 50 4E 47 (\x89PNG)
 - JPEG: FF D8 FF
 - PDF: %PDF or 25 50 44 46
 - ZIP: 50 4B 03 04 (PK^C^D)
 - ELF (executable): 7F 45 4C 46 (\x7FELF)
 - Java bytecode: CA FE BA BE
- Examples
 - `file document.xyz`: Shows actual type regardless of extension
 - `file --mime-type *.txt`: Get MIME types for all text files
 - `file -b image.png`: Brief output showing only file type

- Analyze source code repository: what language is it written in?
 - Create a script that explores a hierarchy of directories containing sources
 - For each file identify its MIME type
 - Count for each type the number of instances
 - Print the list of mime types each with its number of occurrences
- Config file sanitizer: remove comments and empty lined from config files
 - Remove lines that are empty or contain only spaces
 - Remove lines that start with the character #
 - Bonus: remove part of lines from # to the end of the line
 - Must use only one `sed` script
 - Bonus2: write a shell script that identifies *include* statements and process include files recursively

- Log-Based “Report Generator”
 - Analyze apache httpd `access.log` file
 - Find lines with response code of type 404 or 500 (can be done using `grep`)
 - Format the output (can use `sed`) so that for each error it prints:
`Error <RespCode> from IP <IPAddr> at time <timestamp>`
- Basic HTML2MD: convert HTML Web pages into markdown documents
 - Bold conversion: `<b>text</b>` becomes `**text**`
 - Italic conversion: `<i>text</i>` becomes `*text*`
 - Headers: `<h1>text</h1>` becomes `#text`, for other heading `hx` prepend `x` character “#”

Bonus tips and tricks

- Cursor movement (besides the arrows key)
 - **Ctrl + A**: Move the cursor to the start of the line
 - **Ctrl + E**: Move the cursor to the end of the line
 - **Alt + B**: Move the cursor back one word
 - **Alt + F**: Move the cursor forward one word
 - **Ctrl + XX**: Toggle between the start of the line and the current cursor position
- History and Recall
 - **Ctrl + R**: Reverse search. Start typing a part of a previous command to find it in your history. Press **Ctrl + R** again to cycle through matches
 - **Ctrl + G**: Exit the history search mode without running a command
 - **Alt + .** (Period) or **Alt + _**: Insert the last argument of the previous command
 - **!!**: (Not a keyboard shortcut, but a "bang" command) Represents the entire previous command. Use **sudo !!** to re-run a command as root

- Editing and deleting text
 - **Ctrl + U**: Cut (delete) everything from the cursor to the beginning of the line
 - **Ctrl + K**: Cut (delete) everything from the cursor to the end of the line.
 - **Ctrl + W**: Cut the word before the cursor
 - **Alt + D**: Cut the word after the cursor
 - **Ctrl + Y**: "Yank" (paste) the last text you cut using the shortcuts above
 - **Ctrl + T**: Swap the character under the cursor with the one before it (teh → the)
 - **Alt + T**: Swap the current word with the previous one
- Process Control
 - **Ctrl + L**: Clear the screen (similar to the clear command)
 - **Ctrl + C**: Kill (interrupt) the current foreground process
 - **Ctrl + Z**: Suspend the current foreground process
 - **Ctrl + D**: Log out of the current shell or send an EOF (End of File)
 - **Ctrl + S**: Stop output to the screen (freezes the terminal)
 - **Ctrl + Q**: Resume output to the screen (unfreezes)

- 4 modes
 - Command Mode: The default. Used for navigation and manipulation (deleting, copying)
 - Insert Mode: For typing text. (Press **i** to enter)
 - Visual Mode: For selecting blocks of text. (Press **v** to enter)
 - Last Line Mode: For saving, quitting, and global commands. (Press **:** to enter)

- Navigation (mostly Command Mode)
 - `h j k l`: Left, Down, Up, Right
 - `w / b`: Jump to start of next word / back to previous word
 - `<number><motion>`: Move in `<motion>` direction `<number>` times (e.g., `5j` jumps down 5 lines, `5w` jumps 5 words ahead)
 - `0 / $`: Jump to start / end of the current line
 - `gg / G`: Jump to start / end of the file
 - `:number`: Jump to a specific line number (e.g., `:42`) – this occurs actually in last line mode
 - `Ctrl + u / Ctrl + d`: Scroll up / down half a page

- Editing (Command Mode)
 - **i**: Enter Insert mode at the cursor
 - **a**: Append (Insert mode after the cursor)
 - **o**: Open a new line below and enter Insert mode
 - **x**: Delete the character under the cursor
 - **dd**: Delete (cut) the entire current line
 - **<number>d**: Delete **<number>** lines
 - **dw**: Delete to the end of the word
 - **u**: Undo the last action
 - **Ctrl + r**: Redo

- Copy & Paste (Yank & Put)
 - `yy`: Yank (copy) the current line
 - `<number>y`: Yank `<number>` lines starting at current line
 - `y<motion>`: Yank based on movement (e.g., `yw` yanks a word)
 - `p`: Put (paste) after the cursor
 - `P`: Put before the cursor

- Search and Replace
 - `/pattern`: Search forward for a pattern
 - `?pattern`: Search backward for a pattern
 - `n / N`: Repeat search in the same / opposite direction
 - `:%s/old/new/g`: Replace all occurrences of `old` with `new` in the whole file (`g`)
 - `:%s/old/new/gc`: Same as above, but confirm each replacement
- Saving and Quitting (Last Line Mode)
 - `Esc`: Return to Command Mode
 - `:w`: Write (save) the file
 - `:q`: Quit
 - `:wq` or `ZZ`: Save and Quit
 - `:q!`: Quit without saving changes

- Visual Mode (Selection Mode)
 - **v**: Starts Character-wise visual mode. Move the cursor to highlight text
 - **V**: Starts Line-wise visual mode. Selects entire lines at a time
 - **Ctrl + v**: Starts Block-wise visual mode. Useful for selecting columns of text (e.g., deleting the first character of 10 consecutive lines)
- Once text is highlighted
 - **d**: Delete the selection
 - **y**: Yank (copy) the selection
 - **~**: switch the case (upper/lower) of the selected text
 - **>** or **<**: to indent or un-indent the block

- `:set` commands
 - `:set nu`: Show line numbers
 - `:set nonu`: Hide line numbers
 - `:set relativenumber`: Shows line numbers relative to your cursor (great for 5j or 10k jumps)
 - `:set ic`: Ignore case when searching
 - `:set noic`: Make search case-sensitive again
 - `:set hls`: Highlight searches (keeps matches highlighted)
 - `:set nohls`: Temporarily disable search highlighting until next search
 - `:set list`: Show invisible characters (tabs as ^I, end of lines as \$)
 - `:set wrap` / `:set nowrap` Enable or disable line wrapping

- Syntax highlighting
 - `:syntax on`: Enables syntax highlighting based on the file extension (e.g., color-coding for C, Python, or Shell scripts)
 - `:syntax off`: Disables all syntax coloring
 - `:set background=dark` (or `light`): Adjusts the color scheme to be readable against your terminal's background color
 - `:colorscheme <name>`: Changes the entire look of the editor. Common default schemes include desert, evening, blue, and ron
 - `:colorscheme_+Tab` followed by a space and press Tab to cycle through available schemes

- Make configuration permanent using the `~/.vimrc` file
- The file is read and processed at every startup of vim

Example

```
set number  
syntax on  
set hlsearch  
colorscheme desert
```